SMOS

STOCK MANAGEMENT AND ORDERING SYSTEM

Submitted by:

Marri Nagalakshmi (24MIS0278)

Konanki Pranavi (24MIS0325)

Subashini T (24MIS0347)

Submitted to: prof. kamalakannan

Assessment : course based project

Slot: D2+TD2

Introduction

The **Stock Management & Ordering System (SMOS)** is designed to streamline and automate inventory and order management processes within an organization. Traditional inventory systems often rely on manual tracking, which leads to inefficiencies, errors, and delays in stock updates.

This system addresses those issues by providing:

- Real-time inventory tracking
- Automated stock updates
- Efficient supplier and order management

By integrating these functionalities, SMOS minimizes human intervention, reduces errors, and ensures smooth business operations.

Objectives

The primary objectives of the SMOS project are:

- Efficient inventory management
- Automated reorder system
- Supplier optimization
- Secure role-based access
- Scalable system design a
- User-friendly interface.

System Overview

SMOS is a comprehensive system that manages:

- Product inventory
- Supplier information
- Customer orders

It provides a centralized platform where users can:

- View stock levels
- Place orders
- Manage suppliers

The system ensures that all operations are synchronized and updated in real-time, improving operational efficiency.

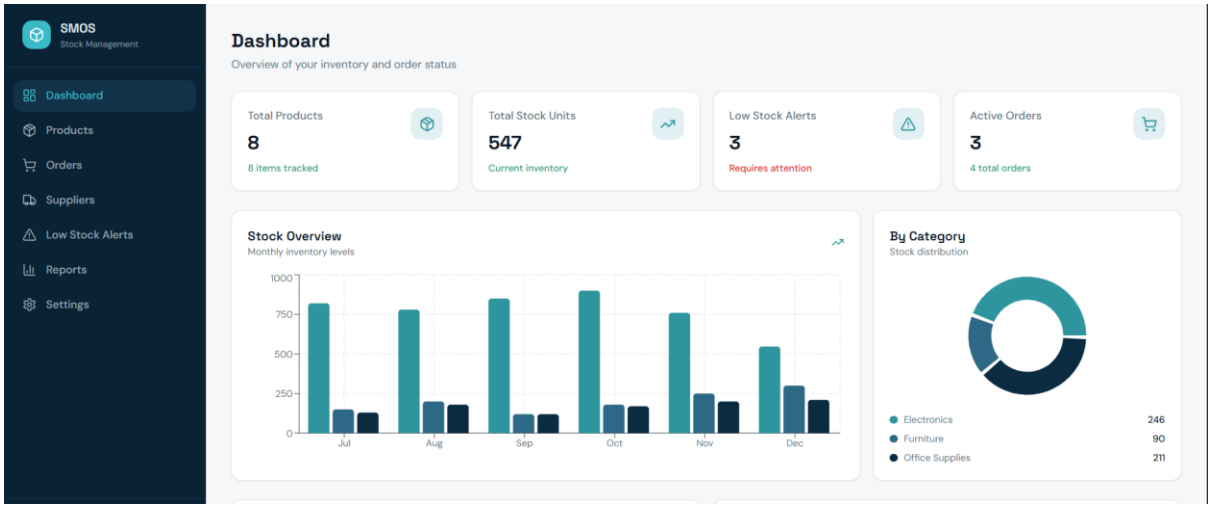

Figure 1 shows dashboard of the SMOS

Development Timeline

The development of the system followed a structured process:

1. **Requirements** gathering: Understanding user needs and defining system requirements.
2. **System Design:** Creating architecture, database schema, and UML diagrams.
3. **Algorithm Selection:** Choosing efficient algorithms for searching, caching, and ordering.
4. **Implementation:** Coding frontend and backend functionalities.
5. **Testing & Debugging:** Identifying and fixing errors to ensure system reliability.

System Workflow

The workflow of SMOS is as follows:

1. **User Login:** Users authenticate using credentials.
2. **Dashboard Overview:** Displays stock levels, alerts, and recent activities.
3. **Product Management:** Add, update, or delete product details.
4. **Order Placement:** Generate and manage purchase orders.
5. **Supplier Handling:** Manage supplier information and performance.
6. **Goods Receipt & Update:** Update stock after receiving goods.

Algorithms Used

The system incorporates several algorithms for efficiency:

- **Threshold-Based Reorder (O(n)): Checks stock levels and triggers reorder when below threshold.**
- **RBAC using HashMap (O(1)): Fast role validation using key-value mapping.**
- **Binary Search (O(log n)): Efficient product lookup in sorted data.**
- **LRU Cache (O(1)): Stores frequently accessed data to reduce database calls.**
- **Observer Pattern (O(k)): Notifies components when stock changes.**
- **Greedy Supplier Selection (O(m log m)): Chooses optimal suppliers based on cost and availability.**

Object-Oriented Analysis & Design (OOAD) Principles

- **Encapsulation Each class (Product, Order, Supplier) hides its internal data behind public methods. For example, stock quantity is only modified through updateStock() and not accessed directly.**
- **Inheritance A base User class is extended by Admin, Manager, and InventoryClerk sub-classes, each inheriting common attributes (userId, name, role) while adding specific behaviour.**
- **Polymorphism The generateReport() method is overridden in StockReport, OrderReport, and LowStockReport classes to produce different output formats from the same interface.**
- **Abstraction The IOrderable interface abstracts the ordering behaviour; both PurchaseOrder and ReturnOrder implement it, hiding implementation details from the calling code.**
- **Single Responsibility Each class has one reason to change – Inventory manages stock data, OrderManager handles order workflow, NotificationService handles alerts.**
- **Open/Closed Principle The ReportGenerator is open for extension (new report types can be added) but closed for modification to existing report logic**

Design Patterns Used

- **Singleton: Ensures a single instance of critical classes.**
- **Factory Pattern: Handles object creation efficiently.**
- **Observer Pattern: Updates dependent modules when stock changes.**
- **Strategy Pattern: Allows dynamic selection of algorithms.**
- **DAO Pattern: Separates database operations.**
- **MVC Architecture: Divides system into Model, View, and Controller.**

System Architecture

- **The system follows a layered (three-tier) architecture to separate concerns and improve maintainability:**
- **Design Patterns used: Singleton (DatabaseConnection), Factory (OrderFactory), Observer (LowStockAlert notifier), Strategy (ReportExport format strategy).**

LAYER	COMPONENT	RESPONSIBILITY
Presentation Layer	Web UI / Desktop GUI	User interaction, input validation, rendering views
Business Logic Layer	Service Classes, Controllers	Core business rules, workflow orchestration, OOAD entities
Data Access Layer	DAO / Repository Classes	Database CRUD operations, query execution
Database Layer	MySQL / PostgreSQL	Persistent storage of products, orders, users, suppliers

Technology Stack

Category	Technology	Purpose
Language	Java / Python	Core application development
Frontend	JavaFX / HTML+CSS	User interface
Backend	Spring Boot / Flask	REST API, business logic
Database	MySQL	Relational data storage
ORM	Hibernate / SQLAlchemy	Object-relational mapping
Version Control	Git + GitHub	Source code management
Build Tool	Maven / pip	Dependency management
Testing	JUnit / pytest	Unit and integration testing
IDE	IntelliJ IDEA / VS Code	Development environment

UML Diagrams

Use case diagram:

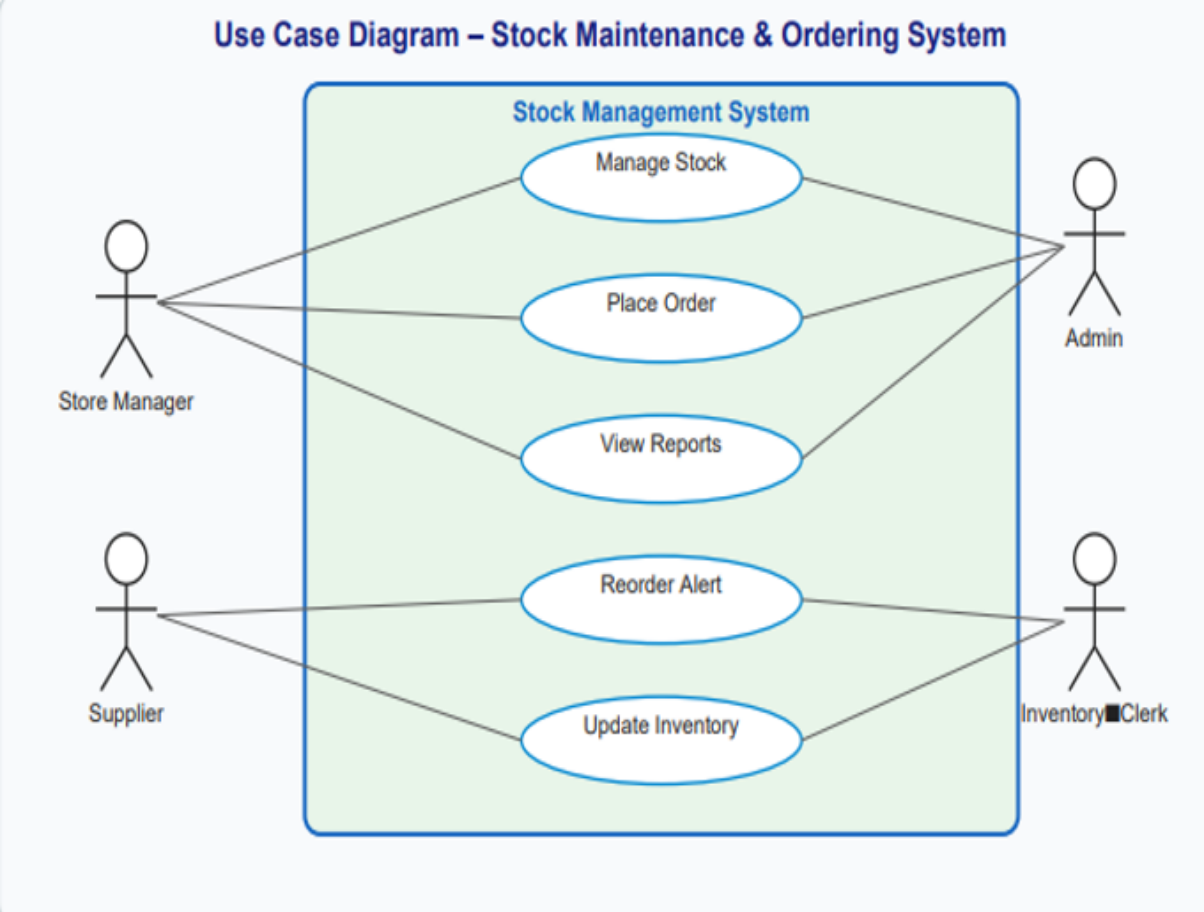

Figure 2 shows the use case diagram for the system. this diagram shows relationship between user and the system

Class diagram:

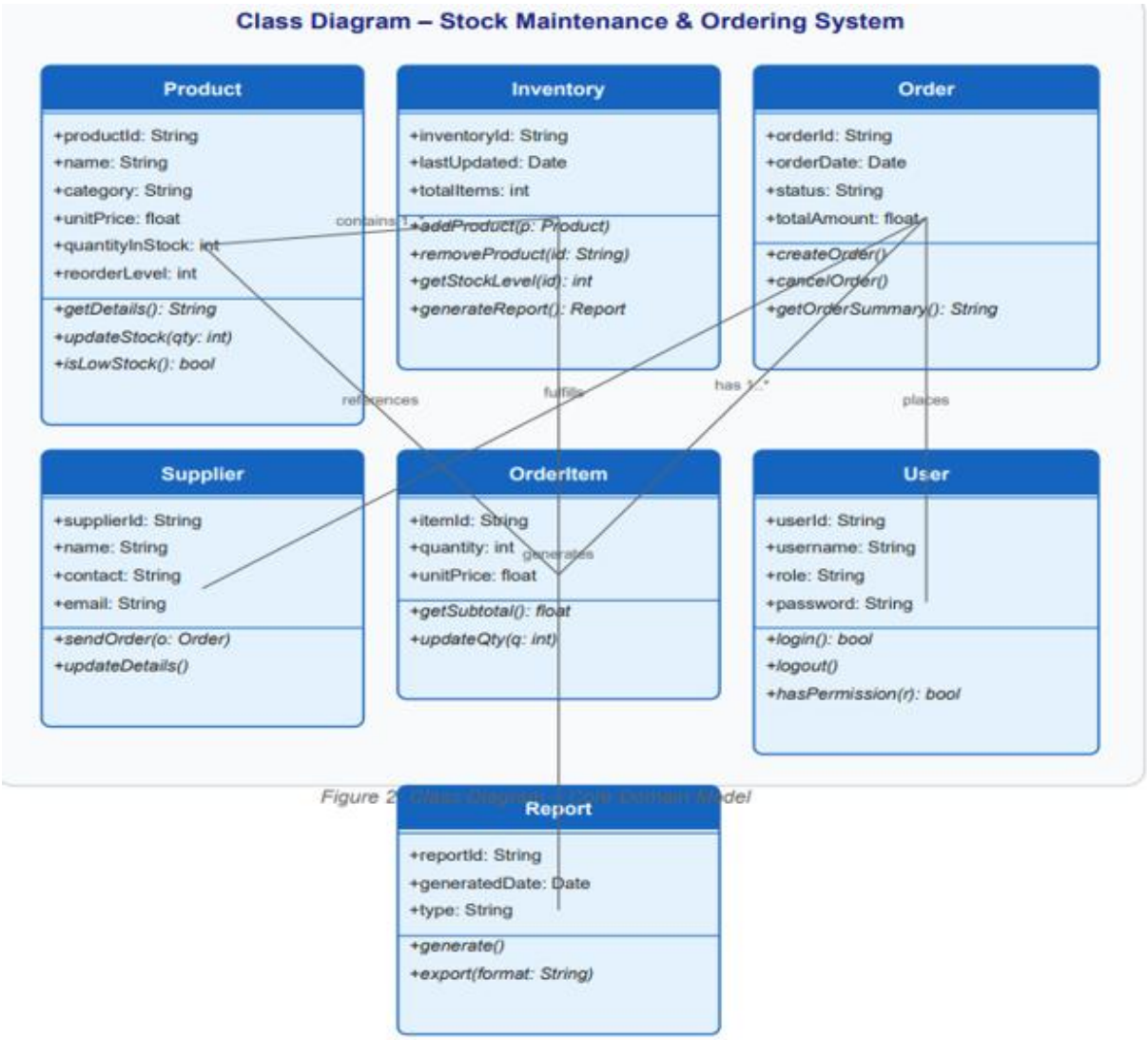

Figure 3shows the class diagram for SMOS

Sequence diagram:

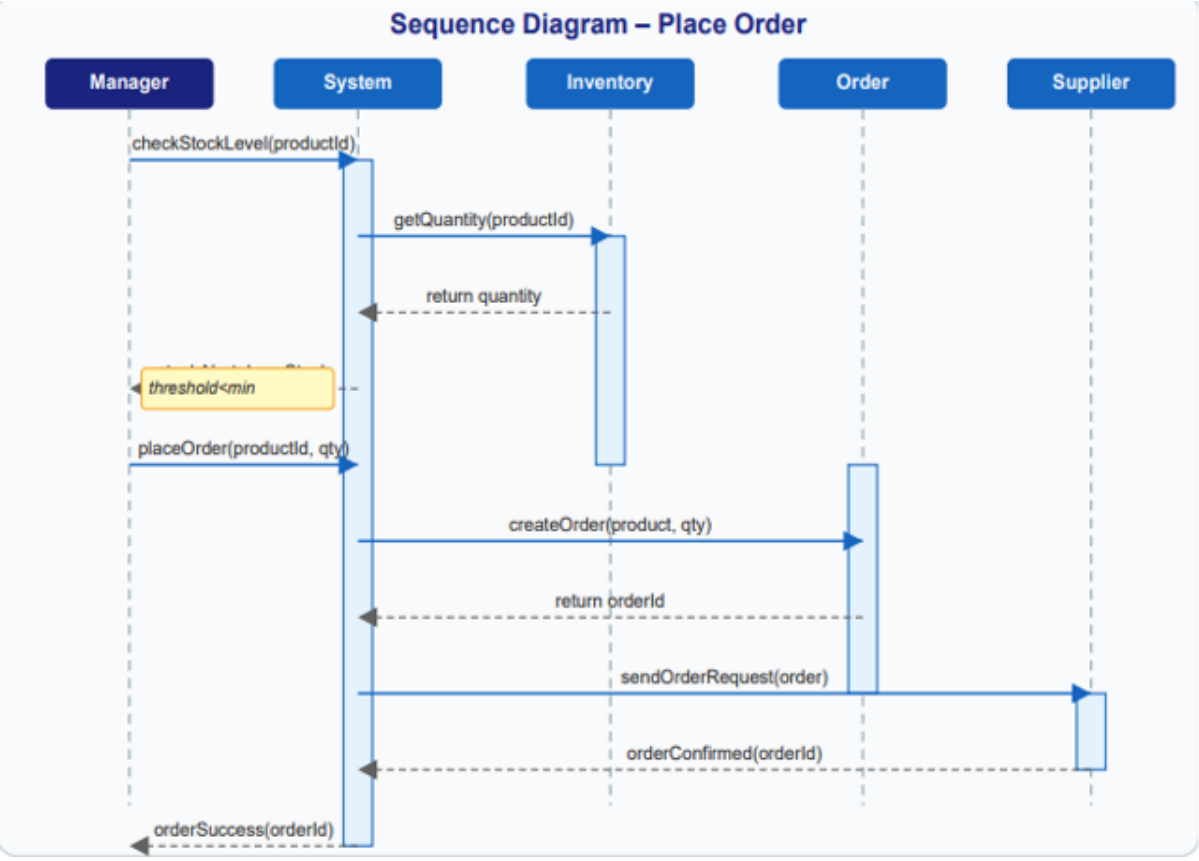

Figure 4shows sequence diagram which indicated the flow of system

Testing Strategy

Test Type	Scope	Tools
Unit Testing	Individual class methods (Product, Order, Inventory)	JUnit 5 / pytest
Integration Testing	Service layer + DAO interactions	Spring Test / Flask-Testing
System Testing	End-to-end workflows (order placement)	Selenium / Manual
UAT	Real-user scenario validation	Manual test scripts
Performance Testing	Load testing with 50 concurrent users	JMeter

Conclusion & Future Scope

Conclusion

The SMOS project successfully provides:

- Efficient inventory control
- Reduced manual effort
- Improved accuracy and performance

Future Scope

- **AI-Based Demand Prediction:**
  Predict stock requirements using machine learning.
- **Cloud Integration:**
  Enable remote access and scalability.
- **Advanced Analytics:**
  Provide insights through dashboards and reports.

RESULT

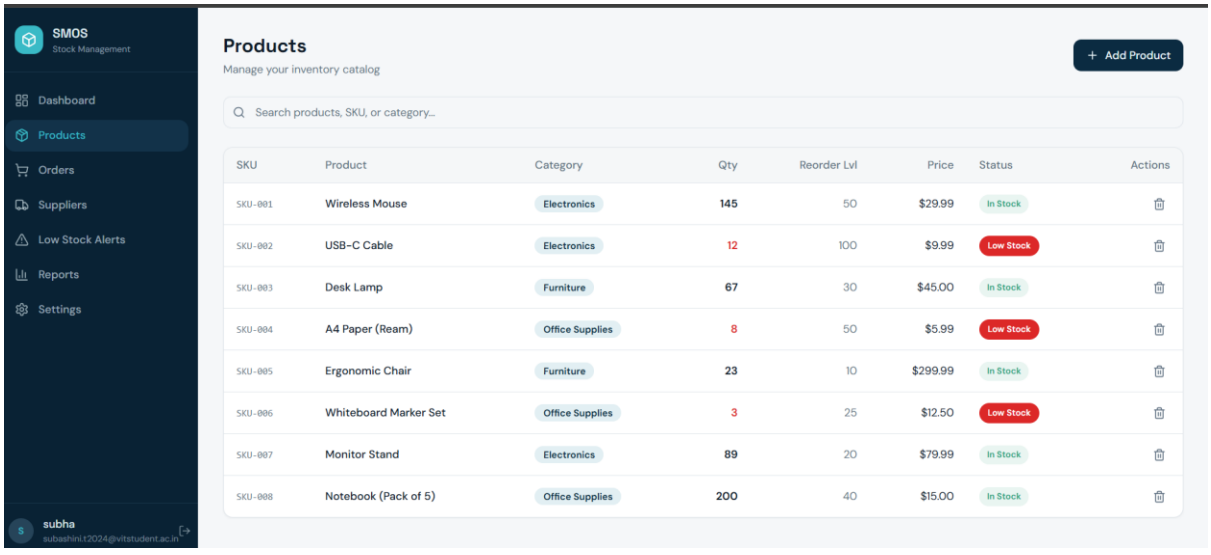

Figure 5shows the product details. by pressing add product we can order many products if it has low stock

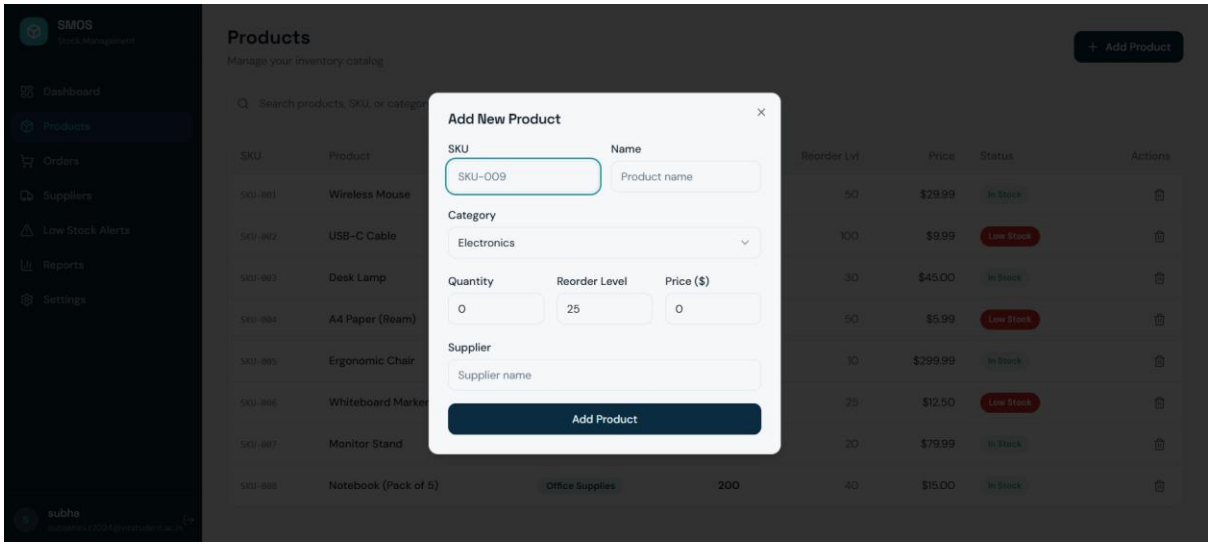

Figure 6shows the interface for add products which have low stock

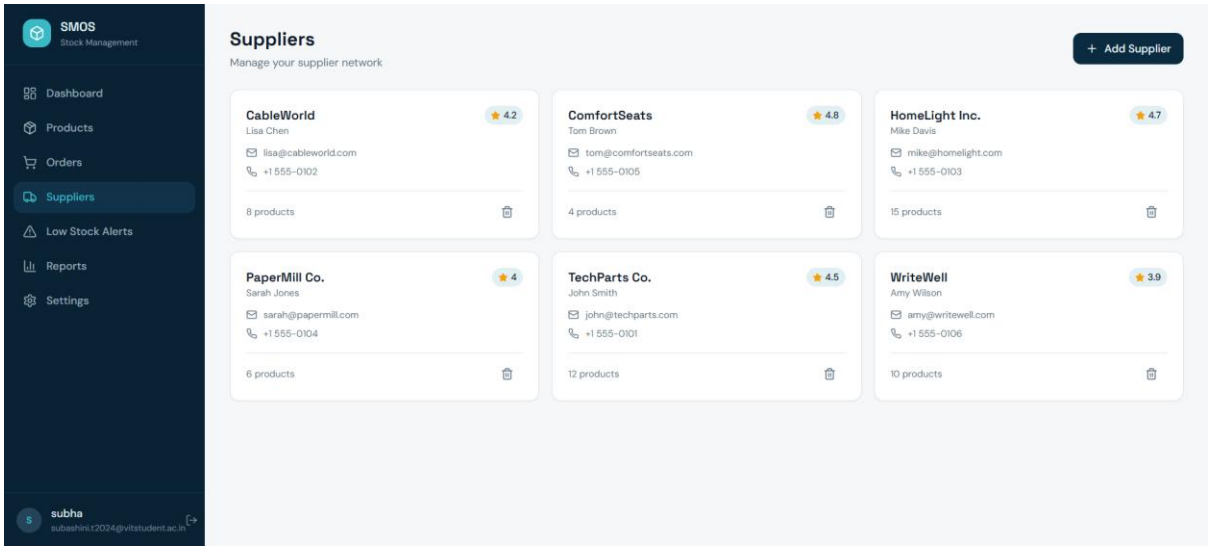

Figure 7shows supplier details with email id and contact details. through add supplier we can add suppliers to order stocks these suppliers are from different domain

Figure 8shows the report which provides detail graph about ordered and received products

REFERENCE

- Software Engineering textbooks
- Online documentation
- Research papers
- Project development guides